



Forbidden Paths

Forbidden Paths



Medium

Web Exploitation

picoCTF 2022

AUTHOR: LT 'SYREAL' JONES

Description

Can you get the flag?

We know that the website files live in `/usr/share/nginx/html/` and the flag is at `/flag.txt` but the website is filtering absolute file paths. Can you get past the filter to read the flag?

Additional details will be available after launching your challenge instance.

This challenge launches an instance on demand.

Its current status is: **NOT_RUNNING**

Launch Instance

Hints ?

(None)

32,576 users solved



88% Liked



picoCTF{FLAG}

Submit Flag

Author: The Analyst: Hyposelenia

Challenge: Forbidden Paths

Category: Web Exploitation

Date: 04/14/26



I. Objective

The objective of this challenge is to **bypass a website's restriction (filter)** and access a hidden file (/flag.txt) to retrieve the flag.

II. Background

Websites often store files on a server. In this challenge, we are told:

- Website files are located in:
/usr/share/nginx/html/
- The flag is located at:
flag.txt

However, the challenge says:

“the website is filtering absolute file paths”

What does that mean?

- **Absolute path** = full path starting from root
Example: /flag.txt **✗** (blocked)

The website is trying to **protect the flag** by blocking direct access to it.

So what is a “Forbidden Path”?

A **forbidden path** is a file path that the website **does not allow you to access directly**.

But sometimes...The filter is weak, and we can **trick it**.

Key Concept: Path Traversal

Path traversal uses ../ which means:

- ../ = go **one folder up**

Example:

- If you are inside:
/usr/share/nginx/html/
- Using ../ moves you to:



/usr/share/nginx/

Using multiple:

- `../..../..../` → keeps going up until you reach the root /

III. Tool Used

- **Web Browser** - Used to access and interact with the challenge website

IV. Methodology

1. Launched the challenge instance

Its current status is: **NOT_RUNNING**

Launch Instance

2. Opened the provided website



3. Observed that there is an input box to read files



Filename

Read

4. Tried using the direct path /flag.txt, but it is **blocked** (filtered)
5. Thought of another way to reach the file

Since the website files are inside:
/usr/share/nginx/html/

We need to go **up directories** first.

6. Entered the following path into the input box:

../../../../flag.txt

Web eReader

..

divine-comedy.txt

oliver-twist.txt

the-happy-prince.txt

../../../../flag.txt

Read

7. Clicked the **Read** button

../../../../flag

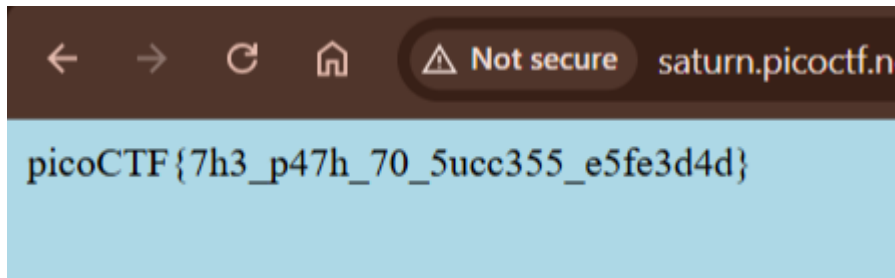
Read

8. The website successfully displayed the contents of the file
9. Retrieved the flag



V. Result

picoCTF{7h3_p47h_70_5ucc355_e5fe3d4d}



VI. Explanation

Let's explain this in the simplest way possible:
Think of folders like this:

```
/(root)
├──usr
│   ├──share
│       ├──nginx
│           └──html ← website starts here
```

The flag is here:

/flag.txt

But the website only allows you to start inside:

/usr/share/nginx/html/

Problem

You are **locked inside a folder**, and the website blocks direct access like:
/flag.txt (forbidden)

Solution (Traversal Trick)



We use:

../

Each ../ moves **one level up**:

- ../ → nginx
- ../../ → share
- ../../.. → usr
- ../../../../ → root /

Now we are at the root, so we can access:

../../../../../flag.txt

Why did this work?

Because:

- The filter only blocks **absolute paths**
- But it does NOT properly block **relative traversal (../)**

This is called a **Path Traversal Vulnerability**

VII. Conclusion

This challenge teaches an important web security concept:

- ✓ Filters can be bypassed if not properly implemented
- ✓ Path traversal (../) can be used to access restricted files
- ✓ Never trust user input without strict validation

Even though the path /flag.txt was “forbidden,” we were still able to reach it by **navigating through directories step-by-step**.

— The Analyst: Hyposelenia